

Laporan Tugas Kecil 1

IF2211 Strategi Algoritma

Penyelesaian Permainan Queens Linkedln Menggunakan Algoritma *Brute Force*



Disusun oleh:

Wisa Ahmaduta Dinutama

18223003

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

Daftar Isi

Daftar Isi.....	2
1. Pendahuluan.....	3
2. Algoritma.....	3
3. Sumber Kode Program.....	4
4. Pengujian.....	9
4. Kesimpulan.....	11
5. Pernyataan.....	11
Lampiran.....	11

1. Pendahuluan

Queens adalah gim logika yang tersedia pada situs jejaring profesional LinkedIn. Tujuan dari gim ini adalah menempatkan queen pada sebuah papan persegi berwarna sehingga terdapat hanya satu queen pada tiap baris, kolom, dan daerah warna. Selain itu, satu queen tidak dapat ditempatkan bersebelahan dengan queen lainnya, termasuk secara diagonal.

Pada tugas kecil ini, permasalahan permainan Queens diselesaikan menggunakan pendekatan algoritma brute force, yaitu metode penyelesaian masalah secara langsung, sederhana, dan naif. Salah satu jenis algoritma *brute force* adalah *exhaustive search* yang melakukan enumerasi terhadap seluruh ruang pencarian yang memungkinkan. Meskipun algoritma ini tergolong menghasilkan kompleksitas waktu yang tinggi, penggunaan *brute force* menjamin penemuan solusi.

2. Algoritma

Algoritma *brute force* yang saya implementasikan merupakan jenis *exhaustive search* dengan ruang pencarian yang sudah diminimalisasi dengan memenuhi batasan satu queen per warna. Pengurangan ruang pencarian disini valid karena tidak mengubah cara kerja algoritma. Pencarian solusi tetap menggunakan *brute force* dengan mengenumerasi semua kemungkinan pada ruang pencarian. Adapun cara kerjanya adalah sebagai berikut.

1. Membaca dan memvalidasi konfigurasi papan dari file input
2. Mengelompokkan seluruh petak papan berdasarkan warna, sehingga setiap warna memiliki senarai berisi posisi petak untuk menempatkan queen
3. Merepresentasikan satu kandidat solusi sebagai kombinasi pemilihan satu petak dari setiap kelompok warna sehingga setiap queen yang dipilih sudah pasti menempati warna yang berbeda antara satu dengan yang lainnya
4. Mengenumerasi seluruh kemungkinan kombinasi tersebut secara sistematis menggunakan mekanisme *increment* indeks kombinasi.
5. Pada setiap kombinasi, program membuat daftar posisi queen yang diuji
6. Memvalidasi konfigurasi dengan memastikan:
 - a. Tidak ada dua atau lebih queen pada baris yang sama
 - b. Tidak ada dua atau lebih queen pada kolom yang sama
 - c. Tidak ada queen yang bersebelahan secara diagonal
7. Jika konfigurasi valid, program diterminasi dan solusi dikembalikan
8. Jika tidak valid, program melanjutkan ke kombinasi berikutnya
9. Jika seluruh kombinasi sudah diperiksa dan tidak ada yang valid, maka program akan menyatakan bahwa solusi tidak ditemukan

3. Sumber Kode Program

Berikut merupakan sumber kode program beserta komentar penjelasannya

```
import tkinter as tk
from tkinter import filedialog, messagebox
import time
try:
    from PIL import Image, ImageDraw
    PIL_INSTALLED = True
except ImportError:
    PIL_INSTALLED = False

COLOR_PALLETES = [
    "#FF6B6B", "#4ECDC4", "#45B7D1", "#96CEB4", "#FFEAA7",
    "#DDA0DD", "#98D8C8", "#F7DC6F", "#BB8FCE", "#85C1E9",
    "#F8C471", "#82E0AA", "#F1948A", "#AED6F1", "#D7BDE2",
    "#A3E4D7", "#FAD7A0", "#A9CCE3", "#D5F5E3", "#FADBD8",
    "#E8DAEF", "#D4E6F1", "#D1F2EB", "#FCF3CF", "#F5B7B1",
    "#ABEBC6",
]

RENDERING_FREQUENCY = 10000

def parse_txt(filepath):
    with open(filepath) as f:
        lines = [line.strip() for line in f if line.strip()]
    return lines

def solve(board_lines, callback=None):
    # dimensi papan
    order = len(board_lines[0])

    # representasi papan pada list satu dimensi
    board = [c for line in board_lines for c in line]
    colors = {}
    for i, color in enumerate(board):
        colors.setdefault(color, []).append(i)

    # list of list yang menyimpan posisi petak pada tiap warna
    colors = list(colors.values())

    NUMBER_OF_QUEENS = len(colors)

    # kombinasi pertama yang menyimpan indeks pertama dari tiap posisi pada tiap warna
    comb = [0 for _ in colors]

    p = NUMBER_OF_QUEENS - 1
    iterations = 0

    while True:
        # membentuk posisi queen berdasarkan kombinasi saat ini
        positions = [colors[color_index][position_index] for color_index, position_index in
                     enumerate(comb)]
        iterations += 1

        # callback untuk visualisasi posisi queen saat ini
        if callback and iterations % RENDERING_FREQUENCY == 0:
            callback(positions, iterations)
```

```

positions_set = set(positions)
rows = set()
cols = set()
is_valid = True

# validasi baris dan kolom
for pos in positions:
    r = pos // order
    c = pos % order
    if r in rows or c in cols:
        is_valid = False
        break
    rows.add(r)
    cols.add(c)

# validasi queen yang bersebelahan secara diagonal
if is_valid:
    is_adjacent_diagonally = False
    for pos in positions:
        r = pos // order
        c = pos % order
        neighbors = [
            (r + 1, c + 1),
            (r + 1, c - 1),
            (r - 1, c + 1),
            (r - 1, c - 1),
        ]
        for nr, nc in neighbors:
            if 0 <= nr < order and 0 <= nc < order:
                if nr * order + nc in positions_set:
                    is_adjacent_diagonally = True
                    break
        if is_adjacent_diagonally:
            break
    if is_adjacent_diagonally:
        is_valid = False

# program diterminasi jika menemukan solusi valid
if is_valid:
    return positions, iterations

# Mencari posisi paling kanan yang masih bisa dinaikkan
# Jika suatu indeks sudah menunjuk ke elemen terakhir pada kelompok warna tersebut,
# maka ia dianggap stuck dan pointer digeser ke kiri
while p >= 0 and comb[p] >= len(colors[p]) - 1:
    p -= 1

# Jika pointer sudah melewati indeks paling kiri,
# berarti semua kombinasi telah dicoba dan tidak ada solusi
if p < 0:
    return None, iterations

# Naikkan indeks pada warna yang sedang ditunjuk pointer
comb[p] += 1

# Semua indeks di sebelah kanan di-reset ke 0
# agar enumerasi kembali dimulai dari kombinasi terkecil
# setelah kenaikan pada posisi tersebut
for i in range(p + 1, NUMBER_OF_QUEENS):
    comb[i] = 0

```

```

        # pointer selalu reset ke warna paling kanan
        p = NUMBER_OF_QUEENS - 1

class App:
    def __init__(self, root):
        self.root = root
        self.root.title("LinkedIn N-Queens Solver")
        self.board_lines = None
        self.solution = None
        self.color_map = {}
        btn = tk.Frame(root)
        btn.pack(pady=5)

        tk.Button(btn, text="Load File", command=self.load_file).pack(side=tk.LEFT, padx=5)
        tk.Button(btn, text="Solve", command=self.run_solve).pack(side=tk.LEFT, padx=5)
        tk.Button(btn, text="Save as Text", command=self.save_txt).pack(side=tk.LEFT, padx=5)
        tk.Button(btn, text="Save as Image", command=self.save_image).pack(side=tk.LEFT,
padx=5)

        self.canvas = tk.Canvas(root, width=400, height=400, bg="white")
        self.canvas.pack(pady=5)
        self.info_var = tk.StringVar(value="Load a board file to begin.")
        tk.Label(root, textvariable=self.info_var).pack(pady=5)

    def load_file(self):
        path = filedialog.askopenfilename(filetypes=[("Text files", "*.txt")])
        if not path:
            return

        try:
            self.board_lines = parse_txt(path)
        except Exception as e:
            messagebox.showerror("Error", str(e))
            return

        if not self.board_lines:
            messagebox.showerror("Error", "Invalid board.")
            return
        N = len(self.board_lines)
        for line in self.board_lines:
            if len(line) != N:
                messagebox.showerror("Error", "Board must be square.")
                return
        colors = set(c for line in self.board_lines for c in line)

        self.color_map = {ch: COLOR_PALLETES[i % len(COLOR_PALLETES)] for i, ch in
enumerate(sorted(colors))}
        self.solution = None
        self.draw_board()
        self.info_var.set("Board loaded. Click Solve.")

    def draw_board(self, queen_positions=None):
        self.canvas.delete("all")
        if not self.board_lines:
            return
        N = len(self.board_lines)
        cell = min(400 // N, 60)
        queen_set = set(queen_positions) if queen_positions else set()

        for r, line in enumerate(self.board_lines):

```

```

        for c, ch in enumerate(line):
            # gambar papan
            x1, y1 = c * cell, r * cell
            x2, y2 = x1 + cell, y1 + cell
            self.canvas.create_rectangle(
                x1, y1, x2, y2,
                fill=self.color_map.get(ch, "gray"),
                outline="black",
            )

            # gambar queen
            if r * N + c in queen_set:
                self.canvas.create_text(
                    (x1 + x2) // 2, (y1 + y2) // 2,
                    text="#", font=("Arial", max(cell // 3, 10), "bold"),
                )

        self.canvas.config(width=N * cell, height=N * cell)

    def on_progress(self, positions, iterations):
        self.draw_board(positions)
        self.info_var.set(f"Iteration: {iterations}")
        self.root.update()

    def run_solve(self):
        if not self.board_lines:
            messagebox.showwarning("Warning", "Load a board first.")
            return

        self.info_var.set("Solving...")
        self.root.update()

        start = time.time()
        result, iterations = solve(self.board_lines, callback=self.on_progress)
        elapsed = (time.time() - start) * 1000

        if result is None:
            self.draw_board()
            self.info_var.set(f"No solution. Time: {elapsed:.2f} ms | Iterations: {iterations}")
        else:
            self.solution = result
            self.draw_board(self.solution)
            self.info_var.set(f"Solved! Time: {elapsed:.2f} ms | Iterations: {iterations}")

    def save_txt(self):
        if self.solution is None:
            messagebox.showwarning("Warning", "Solve the board first.")
            return

        path = filedialog.asksaveasfilename(defaultextension=".txt", filetypes=[("Text files", "*.txt")])
        if not path:
            return

        N = len(self.board_lines)
        queen_set = set(self.solution)
        with open(path, "w") as f:
            for r, line in enumerate(self.board_lines):
                out = ""
                for c, ch in enumerate(line):
                    out += "#" if r * N + c in queen_set else ch

```

```

        f.write(out + "\n")
        messagebox.showinfo("Saved", f"Solution saved to {path}.")

    def save_image(self):
        if self.solution is None:
            messagebox.showwarning("Warning", "Solve the board first.")
            return
        if not PIL_INSTALLED:
            messagebox.showerror("Error", "Pillow (PIL) Library is required to save images.
Refer back to the README to run this app properly.")
            return

        path = filedialog.asksaveasfilename(defaultextension=".png", filetypes=[("PNG",
"*.png")])
        if not path:
            return

        N = len(self.board_lines)
        cell = 60
        queen_set = set(self.solution)

        img = Image.new("RGB", (N * cell, N * cell), "white")
        draw = ImageDraw.Draw(img)
        for r, line in enumerate(self.board_lines):
            for c, ch in enumerate(line):
                x1, y1 = c * cell, r * cell
                x2, y2 = x1 + cell, y1 + cell
                color = self.color_map.get(ch, "#CCCCCC")
                draw.rectangle([x1, y1, x2, y2], fill=color, outline="black")
                if r * N + c in queen_set:
                    draw.text((x1 + cell // 2 - 6, y1 + cell // 2 - 8), "#", fill="black")

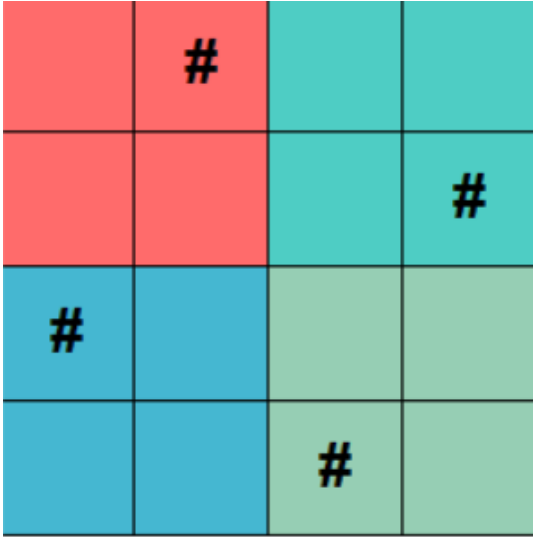
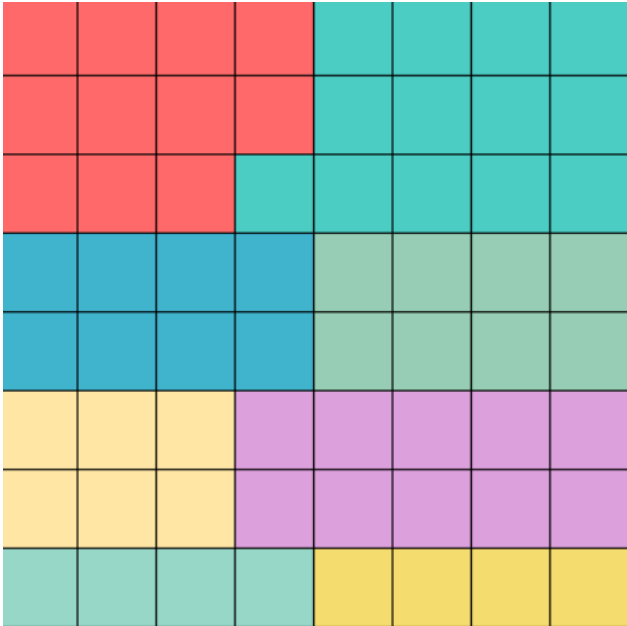
        img.save(path)
        messagebox.showinfo("Saved", f"Image saved to {path}.")

if __name__ == "__main__":
    root = tk.Tk()
    app = App(root)
    root.mainloop()

```


4. Pengujian

Variasi	Input	Output
9x9	AAABBCCCD ABBBBCECD ABBBDCECD AAABDCCCD BBBBDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH	Solved! Time: 54191.16 ms Iterations: 21415356
6x6	AABBBCC ADBBCC DDEBCC DEEFFC DEEFFF DEEFFF	Solved! Time: 13.99 ms Iterations: 9223

4x4	AABB AABB CCDD CCDD	 Solved! Time: 0.22 ms Iterations: 115
Papan tidak valid	AACB AABB CCDD CCDDE	 Error ×  Board must be square. OK
Solusi tidak ditemukan	AAAABBBB AAAABBBB AAABBBBB CCCCDDDD CCCCDDDD EEEEFFFF EEEEFFFF GGGGHHHH	 No solution. Time: 20911.39 ms Iterations: 8785920

4. Kesimpulan

Pada tugas ini telah diimplementasikan algoritma brute force untuk menyelesaikan permainan Queens pada papan berwarna. Pendekatan ini menjamin solusi akan ditemukan jika memang ada, namun waktu eksekusi bergantung pada besar ruang pencarian. Semakin kompleks papan, semakin banyak kombinasi yang harus diuji. Oleh karena itu, pembatasan ruang pencarian berdasarkan batasan permainan dapat membantu meningkatkan efisiensi dari algoritma brute force.

5. Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Wisa Ahmaduta Dinutama

Lampiran

Link Repository

https://github.com/wisauce/Tucil1_18223003

Tabel Pembantu Penilaian

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Program dapat menyimpan solusi dalam bentuk file gambar	✓	